DATA STRUCTURES · DIGITAL NOTES

UNIT III

Stacks

LIFO structure · Push & Pop · Array and linked implementation · Expression evaluation · Backtracking

Roadmap of this unit

3.1	Introduction to stacks: properties & terminology
3.2	Stack operations & the Stack ADT
3.3	Implementing a stack using an array
3.4	Implementing a stack using a linked list
3.5	Applications: expressions, parentheses, reversing, backtracking, recursion

3.1 Introduction to Stacks

DEFINITION

Stack = a linear data structure in which **insertion and deletion happen only at one end**, called the **TOP**. It follows the **LIFO** principle – **L**ast **I**n, **F**irst **O**ut: the element pushed last is the first one to be removed.

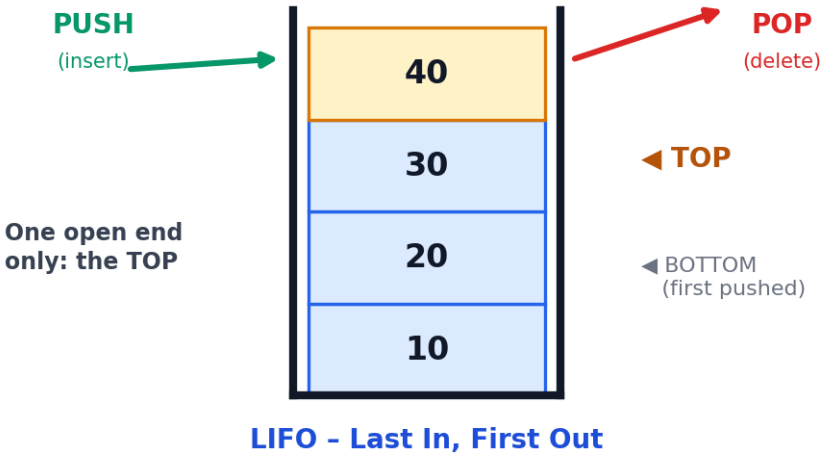


Fig: Stack – a one-ended container; push and pop both work on the TOP

Real-life examples

- A **pile of plates** – you add or take only from the top.
- A **stack of books** or a box of **tennis balls** in a tube.
- **Undo** in a text editor – last change is undone first.
- **Browser Back button** – most recent page opens first.

Properties

- **Linear** and **ordered by time of insertion**.
- **LIFO** (or FILO) access.
- Only the **top** element is accessible – no random access.
- Size can be **fixed** (array) or **dynamic** (linked list).
- Every basic operation takes **O(1)** time.

Terminology

Term	Meaning
TOP	Position of the most recently inserted element (top = −1 means empty stack in array form)
PUSH	Insert an element on the top
POP	Remove and return the top element
PEEK / TOP	Read the top element **without** removing it
Overflow	Trying to push into a **full** stack
Underflow	Trying to pop from an **empty** stack

3.2 Stack Operations and ADT

Operation	Purpose	Time
push(x)	Add x on the top (check overflow first)	O(1)
pop()	Remove top element (check underflow first)	O(1)
peek()	Return top element, stack unchanged	O(1)
isEmpty()	True if stack has no element	O(1)
isFull()	True if no space left (array stack)	O(1)
size()	Number of elements present	O(1)

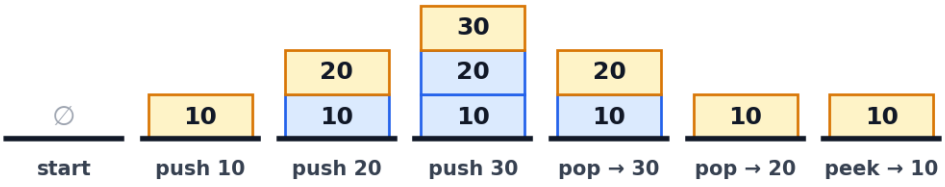


Fig: Sequence of operations – amber cell is always the TOP

Stack ADT specification

Operation	Precondition	Postcondition
push(x)	Stack is not full	x is the new top; size increases by 1
pop()	Stack is not empty	Previous top is removed and returned; size decreases by 1
peek()	Stack is not empty	Top value returned; stack unchanged
isEmpty()	None	Returns true if size = 0

Algorithms

PUSH(x)

1. If **top == MAX − 1** → print “Overflow” and stop.
2. **top = top + 1**
3. **stack[top] = x**

POP()

1. If **top == −1** → print “Underflow” and stop.
2. **x = stack[top]**
3. **top = top − 1**
4. Return x.

Dry run on an array stack (MAX = 3)

Operation	top after	stack[0] [1] [2]	Output / message
(start)	−1	_ _ _	empty
push(10)	0	10 _ _	
push(20)	1	10 20 _	
push(30)	2	10 20 30	stack is now full
push(40)	2	10 20 30	**Overflow**
pop()	1	10 20 (30)	returns 30
pop(), pop()	−1	(10)(20)(30)	returns 20, 10
pop()	−1	_ _ _	**Underflow**

Values in brackets are still in memory but are no longer part of the stack (they are simply overwritten by the next push).

EXAM TIP
In push, **increment first, then store**. In pop, **read first, then decrement**. Always check overflow / underflow **before** changing top.

3.3 Stack using an Array

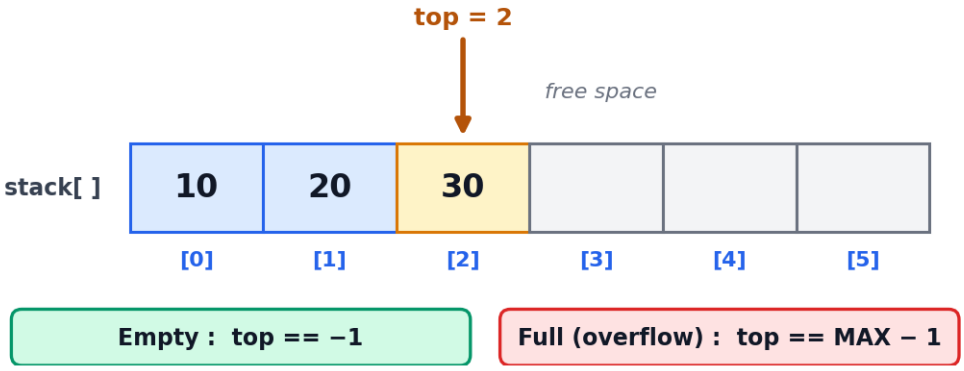


Fig: Array-based stack – top stores the index of the last element

- Use an array **stack[MAX]** and an integer **top** initialised to **−1**.
- Push → **++top** then store. Pop → read then **top−−**.
- Memory is **contiguous**; size is fixed at **MAX**.

```c
#define MAX 5
int stack[MAX];
int top = -1;                                   // empty stack

void push(int x) {
    if (top == MAX - 1) { printf("Stack Overflow\n"); return; }
    stack[++top] = x;                           // increment, then store
}
int pop() {
    if (top == -1) { printf("Stack Underflow\n"); return -1; }
    return stack[top--];                        // read, then decrement
}
int peek()    { return (top == -1) ? -1 : stack[top]; }
int isEmpty() { return top == -1; }
int isFull()  { return top == MAX - 1; }
```

Complete array implementation of a stack

Two stacks in one array

To use space well, two stacks can share **one array**: stack 1 starts at index 0 and grows right; stack 2 starts at the last index and grows left. Overflow happens only when the two tops meet.

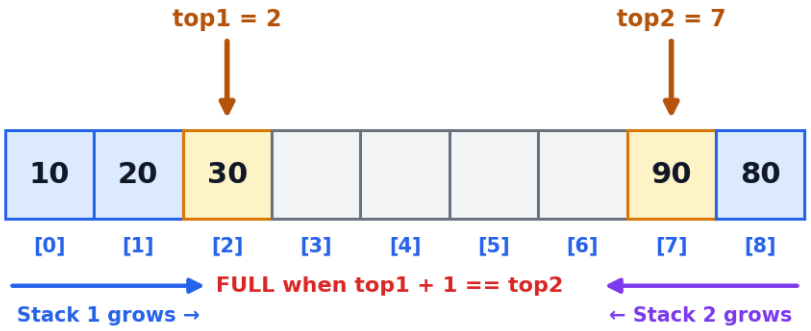


Fig: Two stacks sharing a single array – no space is wasted

Advantages	Disadvantages
Very simple to code	Fixed size – overflow possible
Fast – direct indexing, no pointers	Memory wasted if the stack stays small
Less memory per element	Size cannot grow at run time

3.4 Stack using a Linked List

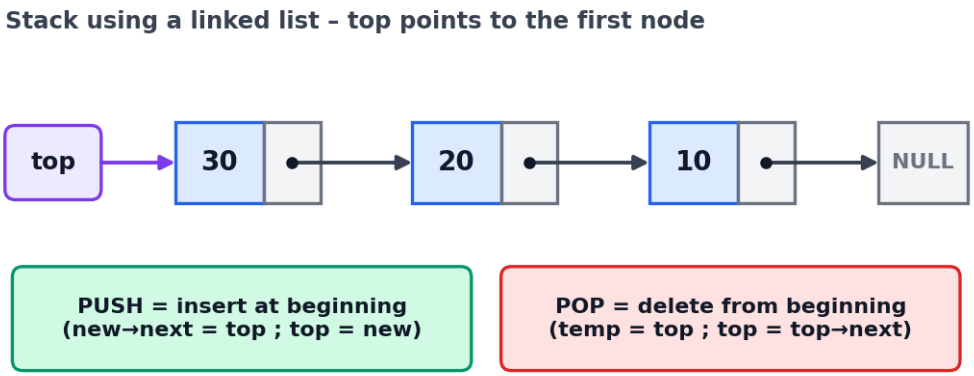


Fig: Linked-list stack – the head of the list acts as the top

- Each element is a **node**; **top** points to the first node.
- **Push = insert at the beginning**, **Pop = delete from the beginning** – both O(1).
- No overflow (until system memory is exhausted); stack is **empty when top == NULL**.

```
struct Node { int data; struct Node *next; };
struct Node *top = NULL;                        // empty stack

void push(int x) {
    struct Node *n = (struct Node*) malloc(sizeof(struct Node));
    n->data = x;
    n->next = top;               // new node points to old top
    top = n;                     // new node becomes top
}
int pop() {
    if (top == NULL) { printf("Stack Underflow\n"); return -1; }
    struct Node *t = top;
    int x = t->data;
    top = top->next;             // move top down
    free(t);
    return x;
}
int peek() { return (top == NULL) ? -1 : top->data; }
```

Basis	Array stack	Linked-list stack
Size	Fixed (MAX)	Dynamic
Overflow	Possible	Almost never
Memory	Contiguous, no pointers	Extra pointer per node
Speed	Slightly faster	Slightly slower (malloc / free)
Use when	Maximum size is known	Size is unpredictable

3.5 Applications of Stacks

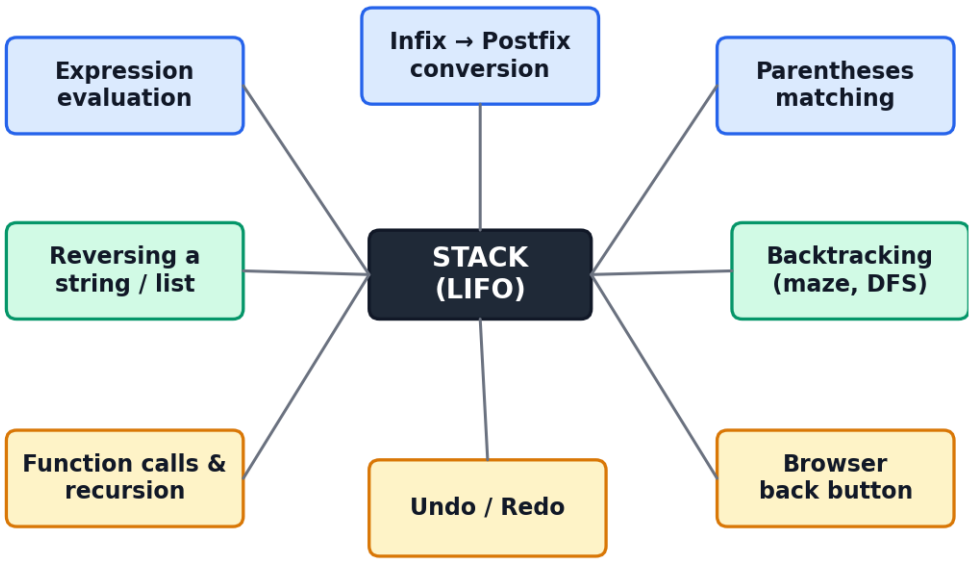


Fig: Where stacks are used

A. Expression evaluation

An expression can be written in three notations. **Computers prefer postfix** because it needs **no brackets** and no precedence rules – it can be evaluated with a single left-to-right scan using a stack.

Notation	Operator position	Example (A + B)	Example: (A + B) * C
Infix	Between operands	A + B	(A + B) * C
Prefix (Polish)	Before operands	+ A B	* + A B C
Postfix (Reverse Polish)	After operands	A B +	A B + C *

Operator	Precedence	Associativity
^ (power)	Highest (3)	Right → Left
*** / %**	2	Left → Right
+ −	Lowest (1)	Left → Right

Infix → Postfix conversion (operator stack)

1. Scan the expression from **left to right**.
2. **Operand** → send directly to output.
3. **“(”** → push on stack.
4. **“)”** → pop to output until “(” is found; discard both brackets.
5. **Operator** → while the stack top has **higher or equal precedence**, pop it to output; then push the current operator.
6. At the end, pop everything left on the stack to output.

Example 1: A + B * C − D → **A B C * + D −**

Symbol	Stack (bottom → top)	Output	Action
A	—	A	operand
+	+	A	push
B	+	A B	operand
*****	+ *	A B	* > + → push
C	+ *	A B C	operand
−	−	A B C * +	pop * and + (≥ −), push −
D	−	A B C * + D	operand
end	—	A B C * + D −	pop all

Example 2: (A + B) * C → **A B + C ***

Symbol	Stack	Output
(	(	—
A	(	A
+	(+	A
B	(+	A B
)	— (pop + until “(”)	A B +
*****	*	A B +
C	*	A B + C
end	—	A B + C *

```
int prec(char c) { return c=='^' ? 3 : (c=='*'||c=='/') ? 2 : (c=='+'||c=='-') ? 1 : 0; }

void infixToPostfix(char in[], char out[]) {
    int k = 0;
    for (int i = 0; in[i]; i++) {
        char c = in[i];
        if (isalnum(c))      out[k++] = c;                          // operand
        else if (c == '(')   push(c);
        else if (c == ')') { while (peek() != '(') out[k++] = pop(); pop(); }
        else { while (!isEmpty() && prec(peek()) >= prec(c)) out[k++] = pop();
               push(c); }                                          // operator
    }
    while (!isEmpty()) out[k++] = pop();
    out[k] = '\0';
}
```

Infix → Prefix conversion

1. **Reverse** the infix expression.
2. Swap every **“(” with “)”** and vice versa.
3. Convert the result to **postfix** (use the same algorithm).
4. **Reverse** the postfix result – this is the prefix.

Example: A + B * C → reverse: C * B + A → postfix: C B * A + → reverse: **+ A * B C**

Evaluating a postfix expression

1. Scan left to right.
2. **Operand** → push.
3. **Operator** → pop **two** operands: first pop = **b** (right), second pop = **a** (left); compute **a op b**; push the result.
4. At the end the single value on the stack is the answer.

Example: 5 6 2 + * 12 4 / − = 5 × (6 + 2) − 12 ÷ 4 = **37**

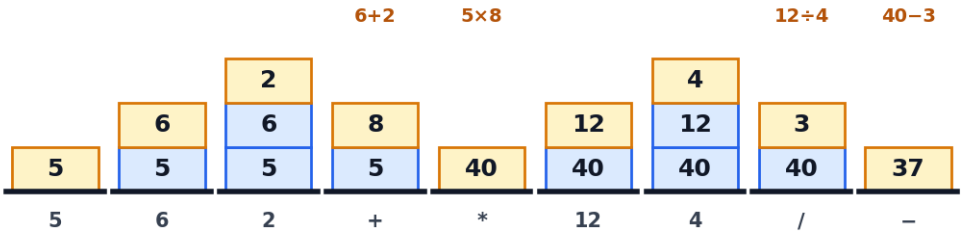


Fig: Stack contents after each symbol of the postfix expression

```
int evalPostfix(char e[]) {
    for (int i = 0; e[i]; i++) {
        if (isdigit(e[i])) push(e[i] - '0');                  // operand
        else {
            int b = pop(), a = pop();                         // note the order: b first
            switch (e[i]) { case '+': push(a + b); break;  case '-': push(a - b); break;
                            case '*': push(a * b); break;  case '/': push(a / b); break; }
        }
    }
    return pop();
}
```

More practice (solved):

Infix	Postfix	Prefix
A + B * C	A B C * +	+ A * B C
(A + B) * (C − D)	A B + C D − *	* + A B − C D

Infix	Postfix	Prefix
A * B + C / D	A B * C D / +	+ * A B / C D
A − B − C	A B − C − (left to right)	− − A B C

COMMON MISTAKE
In subtraction and division the **order matters**: result = (second popped) **op** (first popped). Writing b − a instead of a − b is the most common error.

B. Parenthesis matching (balanced brackets)

- Scan the string; **push every opening** bracket ([{.
- On a **closing** bracket, pop and check that it is the **matching** opening bracket (stack empty → unbalanced).
- At the end the stack must be **empty**.

Expression	Process	Result
{ [( )] }	push { [(→) matches (→] matches [→ } matches { → stack empty	**Balanced**
([ )]	push ([→) meets [on top → mismatch	**Not balanced**
(( )	push ((→) matches → one “(” left	**Not balanced**

```
int isBalanced(char s[]) {
    for (int i = 0; s[i]; i++) {
        char c = s[i];
        if (c == '(' || c == '[' || c == '{') push(c);                  // opening
        else if (c == ')' || c == ']' || c == '}') {
            if (isEmpty()) return 0;                                    // nothing to match
            char o = pop();
            if ((c == ')' && o != '(') || (c == ']' && o != '[') ||
                (c == '}' && o != '{')) return 0;
        }
    }
    return isEmpty();                                       // must be empty at end
}
```

C. Reversing a string / list

Push every character, then pop them one by one. Because of LIFO, the order comes out **reversed**. Same idea reverses a linked list or an array.

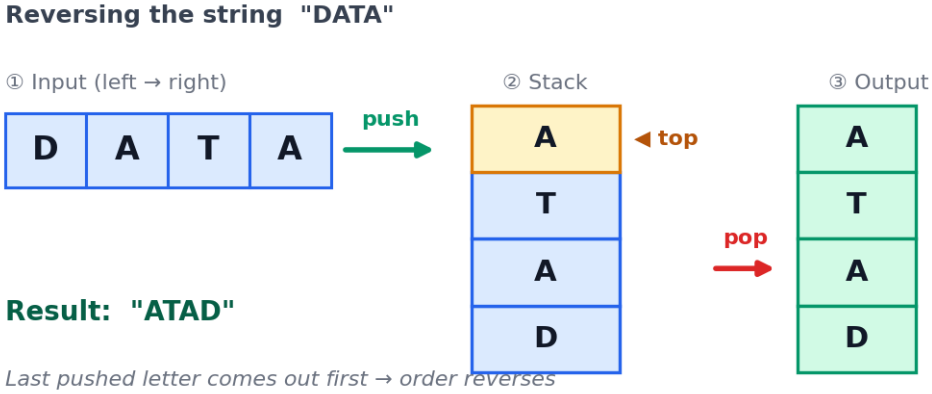


Fig: Reversing “DATA” with a stack

```
void reverse(char s[]) {
    for (int i = 0; s[i]; i++) push(s[i]);          // push all characters
    for (int i = 0; s[i]; i++) s[i] = pop();        // pop them back -> reversed
}
```

- **Palindrome check:** reverse with a stack and compare with the original.

D. Backtracking

Backtracking = try a choice; if it leads to a dead end, **undo it (pop)** and try the next choice. The stack remembers the path taken so far.

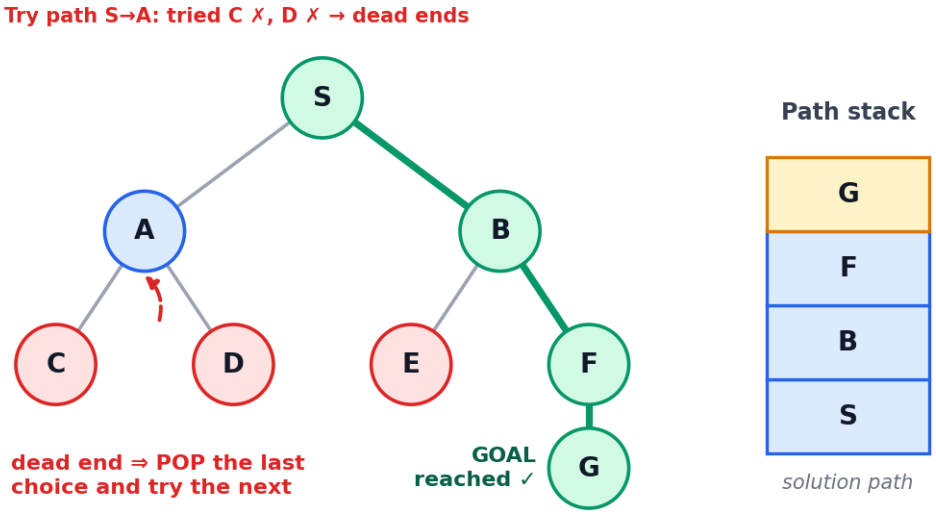


Fig: Backtracking in a maze / decision tree – dead ends are popped from the path stack

- **Push** a position when you move forward.
- **Pop** when there is no unvisited neighbour (dead end).
- When the goal is reached, the stack holds the **solution path**.
- Used in: **maze solving, N-Queens, Sudoku, depth-first search (DFS)** in graphs.

E. Function calls and recursion

- Every function call pushes an **activation record (stack frame)** – parameters, local variables and return address.
- When the function returns, its frame is **popped** and control goes back to the caller.
- Deep or infinite recursion → **stack overflow** error.

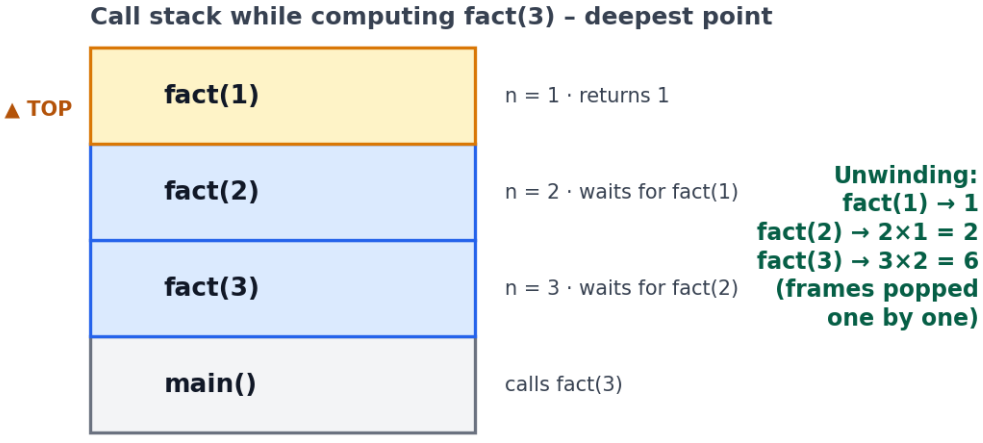


Fig: System call stack for fact(3)

F. Decimal to binary conversion

Divide the number by 2 repeatedly and **push each remainder**. The **last remainder is the most significant bit**, so popping gives the binary number in the correct order.

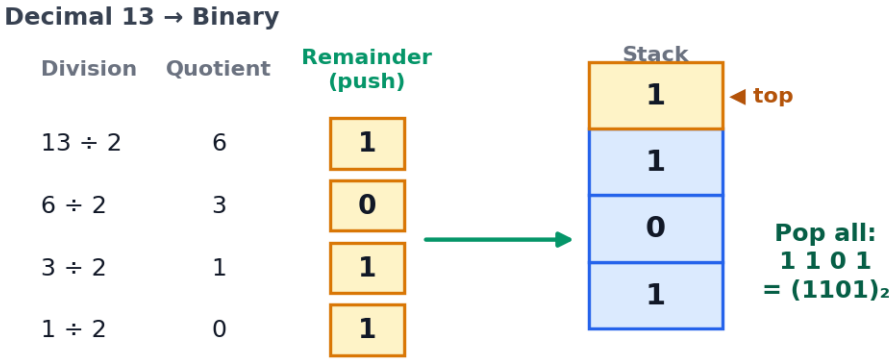


Fig: Converting 13 to binary using a stack

```
void toBinary(int n) {
    while (n > 0) { push(n % 2); n = n / 2; }     // push remainders
    while (!isEmpty()) printf("%d", pop());       // pop -> correct order
}
```

G. Other applications

- **Undo / Redo** – two stacks of actions.
- **Browser history** – Back button pops the last page.
- **Converting decimal to binary** – push remainders, pop to print.
- **Syntax checking** by compilers (brackets, begin–end).
- **Memory management** – local variables are stored on the stack.

3.6 Quick Revision & Practice

ONE-PAGE SUMMARY

- Stack = **LIFO**; all operations at the **TOP**; push, pop, peek are **O(1)**.
- Array stack: top = −1 empty, top = MAX−1 full. Linked stack: top = NULL empty.
- Postfix needs **no brackets**; evaluate with an operand stack; convert with an operator stack.
- Reversal, bracket matching, backtracking and recursion all rely on LIFO.
- Overflow = push on full. Underflow = pop on empty.

Likely exam questions

2 marks

- Define stack. Why is it called LIFO?
- What are overflow and underflow?
- Convert (A+B)*C to postfix.
- List any four applications of stacks.

5 marks

- Write algorithms for PUSH and POP using an array.
- Compare stack implementation using array and linked list.
- Explain how a stack is used to check balanced parentheses.
- Evaluate the postfix expression 2 3 1 * + 9 − using a stack.

10 marks

- Explain infix-to-postfix conversion with algorithm, example and program.
- Explain stack ADT with array and linked-list implementations and applications.

Try it yourself

1. Convert **A * (B + C) / D** to postfix. *(Answer: A B C + * D /)*
2. Evaluate **8 2 3 ^ / 2 3 * + 5 −** where ^ is power. *(Answer: 8 ÷ 8 + 6 − 5 = 2)*
3. Is **{ ([ ]) })** balanced? Show the stack.
4. Write a program to convert a decimal number to binary using a stack.